SIMATS ENGINEERING

ASSESSMENT TOOL 2 : Scenario-Based

COURSE NAME: Cloud Computing and Big Data Analytics **COURSE CODE:** CSA15

COURSE OUTCOME COVERED

CO2: *Analyze virtualization architectures to identify performance and security issues in cloud computing environments.* (BL3)

Total Marks: 25

Dave's Taxonomy: Manipulation (Level 2)
Question Count: 5

Code of Conduct

I Abdul Riyaz Ahammad (192525106) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.
Signature of the student: Abdul Riyaz Ahammad

Assessment Tasks
Scenario-Based Questions

1. A media company wants to migrate its video streaming platform to the cloud.
   The system must handle millions of concurrent users with minimal buffering.
   Design a cloud architecture using scalability and caching techniques.
   Explain how load balancing and auto-scaling can ensure smooth streaming.
   Discuss cost optimization strategies for such high-demand workloads.

2. A fintech startup is deploying applications using containers across multiple cloud providers.
   They face challenges in maintaining consistency and managing deployments.
   Propose a solution using container orchestration and multi-cloud strategies.
   Explain how service discovery and scaling can be handled efficiently.
   Highlight the benefits and risks of a multi-cloud deployment model.

3. An e-learning platform experiences data loss due to accidental deletion in the cloud.
   The organization needs a robust backup and disaster recovery strategy.
   Design a solution using cloud-native backup, replication, and versioning.

Explain how recovery time objective (RTO) and recovery point objective (RPO) are achieved.
Provide steps to ensure business continuity during failures.

4. A government agency is adopting a hybrid cloud model for sensitive and non-sensitive data.
   Some applications must remain on-premises due to compliance requirements.
   Design a hybrid architecture that ensures secure communication between environments.
   Explain how identity management and encryption should be implemented.
   Discuss challenges in managing hybrid cloud environments.

5. A SaaS provider needs to ensure high availability and fault tolerance for its application.
   The system must continue operating even if one region goes down.
   Propose a multi-region deployment strategy in the cloud.
   Explain how data replication and failover mechanisms should be configured.
   Discuss monitoring and alerting strategies for proactive issue detection.

Scenario-Based Rubric

Criteria	**Weightage**	**Level 4 - Exemplary**	**Level 3 - Proficient**	**Level 2 - Developing**	**Level 1 - Beginning**
Understanding of Scenario	20%	Demonstrates complete understanding of the scenario context	Good understanding with minor gaps	Partial understanding	Fails to understand the scenario
Identification of Risks / Issues	20%	Identifies all major issues correctly	Identifies most issues	Limited issue identification	Misses major issues
Application of Concepts	25%	Applies virtualization concepts effectively	Applies concepts with minor mistakes	Partially correct application	Weak application
Decision Justification	20%	Decisions are well justified and technically strong	Reasonable justification	Weak justification	No useful justification
Clarity	15%	Clear and well-organized response	Generally clear	Somewhat unclear	Poorly presented

SET 2:

1.A retail company collects terabytes of customer transaction data every day from online and offline stores. The management wants to analyze purchasing patterns in near real time to improve product recommendations and inventory management.

- Design a cloud-based big data architecture using distributed storage and processing frameworks.
- Explain how batch processing and stream processing can be integrated.
- Discuss how data lakes and data warehouses can be used together.
- Recommend techniques to ensure scalability and cost efficiency.

2.A healthcare organization is migrating patient records and medical imaging data to the cloud. The solution must comply with strict privacy regulations while allowing authorized doctors to access records securely from multiple locations.

- Design a secure cloud architecture that protects sensitive healthcare data.
- Explain how encryption, identity and access management (IAM), and key management should be implemented.
- Discuss auditing, logging, and compliance monitoring strategies.
- Recommend methods to prevent unauthorized access and data breaches.

3. An online ticket booking platform experiences sudden spikes in traffic whenever popular events are announced. The company wants to reduce infrastructure management while ensuring fast response times.

- Design a serverless cloud architecture for the application.
- Explain how event-driven computing can improve scalability.
- Discuss the role of managed databases, API gateways, and message queues.
- Evaluate the advantages and limitations of serverless computing for this workload.

4. A multinational company gathers data from sales, finance, manufacturing, and customer support systems. The organization requires a centralized analytics platform with high-quality and consistent data.

- Design a cloud-based ETL/ELT pipeline for integrating data from multiple sources.
- Explain how data quality, metadata management, and governance should be maintained.
- Discuss the role of workflow orchestration and scheduling tools.
- Recommend strategies for optimizing query performance and storage costs.

5. A logistics company wants to predict delivery delays using historical shipment data, weather information, and live GPS tracking. The analytics platform must support continuous model training and real-time predictions.

- Design a cloud-based machine learning and analytics architecture.
- Explain how big data technologies can support model training and inference.
- Discuss the use of cloud-native AI/ML services for model deployment and monitoring.
- Recommend strategies for handling large-scale data ingestion, feature engineering, and model updates.

SET 3 :

1. A manufacturing company plans to migrate its legacy enterprise applications and databases from an on-premises data center to the cloud. The organization wants to minimize downtime while ensuring application performance and security.

- Design a cloud migration strategy suitable for the organization.
- Explain how different migration approaches (Rehost, Refactor, Replatform, Repurchase, Retire, Retain) can be applied.
- Discuss methods to validate application performance after migration.
- Recommend strategies to reduce migration risks and downtime.

2. A smart city project deploys thousands of IoT sensors to monitor traffic, air quality, energy consumption, and public safety. The collected data must be processed and analyzed in real time for decision-making.

- Design a cloud-based architecture for IoT data collection and analytics.
- Explain how message brokers, stream processing, and cloud storage are integrated.
- Discuss methods for handling high-volume, high-velocity sensor data.
- Recommend techniques to ensure scalability, reliability, and security.

3. A global e-commerce company wants executives to monitor sales performance, customer behavior, and regional trends through interactive dashboards. The solution must support real-time analytics and historical reporting.

- Design a cloud-based business intelligence architecture.
- Explain how data visualization tools integrate with cloud data warehouses.
- Discuss techniques for optimizing dashboard performance with large datasets.
- Recommend security measures to protect business reports and sensitive data.

4. An online gaming company experiences fluctuating workloads throughout the day. During peak hours, users report high latency and reduced application performance.

- Design a cloud monitoring and performance optimization solution.

- Explain how cloud monitoring tools, metrics, logs, and tracing help identify performance bottlenecks.
- Discuss strategies for resource optimization and capacity planning.
- Recommend methods to improve application availability while controlling operational costs.

5. A financial institution stores petabytes of customer transaction data for regulatory compliance and business analytics. The organization must retain historical records while managing storage costs efficiently.

- Design a cloud-based data lifecycle management strategy.
- Explain how data classification, retention policies, and archival storage should be implemented.
- Discuss the importance of governance, auditing, and regulatory compliance in big data environments.
- Recommend techniques for balancing data accessibility, security, and cost optimization.

ANSWERS

Q1. Video Streaming Platform for Millions of Concurrent Users

1. Understanding of the scenario

A media company is moving a video streaming service to the cloud. The workload is read-heavy, bandwidth-heavy and extremely spiky (live events, new releases). Success means low startup delay, minimal re-buffering and predictable cost at millions of concurrent viewers.

2. Risks and issues identified

- **Bandwidth and origin overload:** all viewers hitting the origin would saturate network and storage.
- **Traffic spikes:** sudden surges cause slow scale-up, so buffering and failed sessions.
- **Latency for distant users**, uneven network quality, and a single-region failure.
- **Cost explosion:** egress bandwidth and over-provisioned compute dominate the bill.

3. Application of concepts: proposed architecture

- **Content storage and delivery:** videos transcoded into adaptive-bitrate (ABR) HLS/DASH segments (about 4 s each) and stored in object storage; a multi-tier **CDN** (edge cache → origin shield → origin) serves segments close to users.
- **Caching techniques:** CDN edge caching with long TTL for segments and short TTL for manifests, origin shield to collapse duplicate requests, Redis for sessions/metadata/recommendations, and browser/player buffering.
- **Load balancing:** GeoDNS/latency routing directs users to the nearest healthy region; an L7 load balancer with health checks spreads requests across origin, packager and API instances in multiple availability zones; connection draining avoids dropped streams during scale-in.
- **Auto-scaling:** stateless services in auto-scaling groups (or Kubernetes HPA) scaled by target-tracking on CPU/network throughput, plus scheduled scaling before known events and predictive scaling; transcoding workers scale on queue length.
- **Smooth playback:** ABR lets the player drop to a lower bitrate rather than buffer; a startup buffer of about 3 segments (12 s) absorbs jitter.

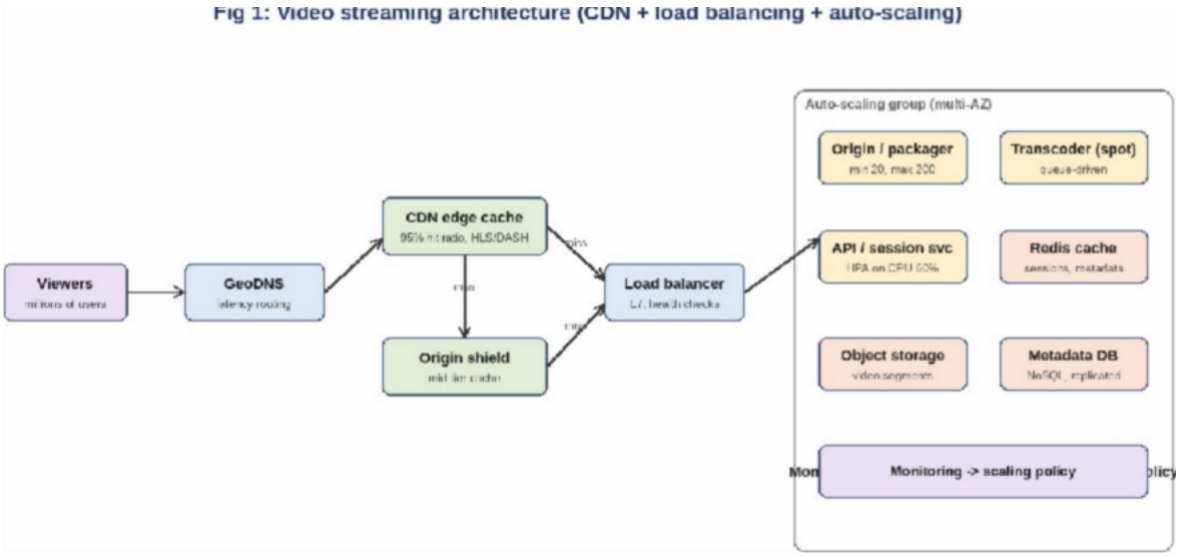


Figure 1: Streaming architecture with CDN caching, load balancing and auto-scaling.

4. Calculations (assumed values)

Item	Calculation and result
Peak egress	5,000,000 users × 5 Mbps = 25,000,000 Mbps = **25 Tbps**
Origin load with CDN	95% hit ratio → 5% × 25 Tbps = **1.25 Tbps**
Origin servers needed	10 Gbps NIC at 70% usable = 7 Gbps; 1.25 Tbps / 7 Gbps = 178.6 → **≈ 179 servers**. Without CDN: 25 Tbps / 7 Gbps = 3,572 servers, so the CDN removes about **95%** of origin capacity.
Data per viewer-hour	5 Mbps × 3,600 s = 18,000 Mb = 2,250 MB = **2.25 GB**
Hourly delivery volume	5,000,000 × 2.25 GB = 11,250,000 GB ≈ **11.25 PB/hour**
CDN cost at $0.02/GB (committed)	11,250,000 × 0.02 = **$225,000/hour**
Saving from smarter ABR ladder (average 5 → 3.5 Mbps, 30% less data)	$225,000 × 0.70 = $157,500, saving **$67,500/hour**
Auto-scaling rule	desired = ceil(current × metric / target). 100 instances at 85% CPU, target 60%: 100 × 85 / 60 = 141.7 → **142 instances**
Spot transcoding	1,000 instances × 10 h × $0.20 = $2,000 on-demand; at 70% spot discount = $600, saving **$1,400 per batch**

5. Cost optimization and decision justification

- **Reserved/committed capacity for the steady baseline, on-demand for peaks, spot for transcoding** (fault-tolerant, restartable jobs).
- **CDN and egress:** multi-CDN with committed-spend discounts, high cache-hit ratio, modern codecs (HEVC/AV1) and per-title encoding to lower bitrate at equal quality.
- **Storage tiering:** hot titles on standard storage, long-tail content on infrequent-access/archive via lifecycle rules.
- **Scale-in policies and right-sizing**, plus budgets, tagging and cost anomaly alerts.

Justification: a CDN is chosen because the calculation shows it removes about 95% of origin load and is the only economical way to deliver 25 Tbps; auto-scaling is chosen over fixed capacity because demand varies widely, and a mixed purchasing model cuts cost without risking playback quality.

Q2. Containers Across Multiple Cloud Providers (Fintech Startup)

1. Understanding of the scenario

A fintech startup runs containerised services on several cloud providers. Each provider has its own tools, networking and IAM, so configuration drifts, releases are inconsistent and operations are hard. The need is one consistent, secure, repeatable way to deploy, discover and scale services everywhere.

2. Risks and issues identified

- **Configuration drift** and inconsistent environments between providers.
- **Manual, error-prone deployments**; no single view of what is running where.

- **Security and compliance gaps** (different IAM models, secrets handling); fintech data is regulated.
- **Cross-cloud networking, latency and egress cost**; service-to-service discovery across clusters.

3. Application of concepts: proposed solution

- **Container orchestration:** Kubernetes on each provider (EKS/AKS/GKE) so the same API and manifests work everywhere; Helm/Kustomize for packaging; a replicated container registry with image scanning and signing.
- **Consistency:** infrastructure as code (Terraform) plus **GitOps** (Argo CD/Flux): Git is the single source of truth and controllers reconcile each cluster to it; policy-as-code (OPA/Kyverno) enforces the same security rules; Vault for secrets.
- **Multi-cloud strategy:** a cloud-agnostic application layer (containers, open standards), a fleet/multi-cluster manager, a global load balancer/DNS for traffic steering, and centralised observability.
- **Service discovery:** CoreDNS and Kubernetes Services inside a cluster; a service mesh (Istio/Linkerd) with multi-cluster discovery for cross-cloud calls; mTLS between services and health-based routing.
- **Efficient scaling:** Horizontal Pod Autoscaler on CPU/custom metrics, KEDA for event-driven scaling, Cluster Autoscaler for nodes, and resource requests/limits to avoid noisy neighbours.

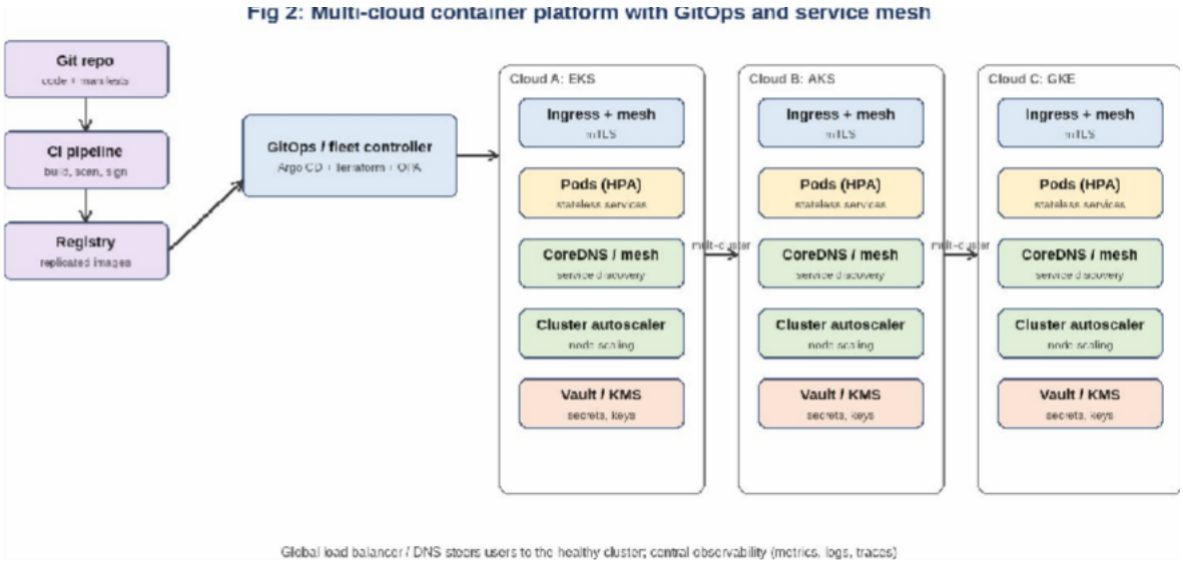


Figure 2: Multi-cloud Kubernetes platform managed through GitOps with a service mesh.

4. Calculations (assumed values)

Item	Calculation and result
Availability, single cloud (99.95%)	Downtime = 0.0005 × 8,760 h = **4.38 h/year**
Availability, two clouds active-active	Unavailability = $0.0005^2 = 2.5 \times 10^{-7}$ → **99.999975%**; downtime = 8,760 × 3,600 × 2.5×10^{-7} ≈ **7.9 s/year** (theoretical upper bound; shared DNS/CI dependencies lower the real value)

Item	Calculation and result
HPA scale-out	desired = ceil(current × metric / target): 10 pods at 90% CPU, target 60% → 10 × 90 / 60 = **15 pods**; at 30% → 10 × 30 / 60 = **5 pods**
Cross-cloud egress cost	20 TB/month = 20,480 GB × $0.09 = **$1,843/month**; keeping chatty services in the same cloud and compressing cuts about 60% → **$737/month**
Spare capacity to survive losing one cloud	2 clouds each sized for 100% = 200% provisioned; 3 clouds each sized for 50% = **150% provisioned** (25% cheaper)

5. Benefits and risks of multi-cloud; decision justification

Benefits	Risks
Avoids vendor lock-in; stronger negotiating position	Higher operational complexity; need for broad skills
Resilience: survive a provider or region outage	Larger attack surface; inconsistent IAM/security tooling
Choose best service or region for compliance, latency, price	Data egress charges and cross-cloud latency
Regulatory data-residency options for fintech	Lowest-common-denominator design may skip useful managed services
Burst capacity from another provider	Harder cost tracking, monitoring and debugging across clouds

Justification: Kubernetes plus GitOps is chosen because it gives identical, auditable deployments on every provider with rollback through Git. A service mesh gives secure, discoverable service-to-service traffic without code changes. Multi-cloud is accepted because the calculation shows a large availability gain, with risks managed through policy-as-code, central observability and keeping chatty services close together.

Q3. Backup and Disaster Recovery for an E-Learning Platform

1. Understanding of the scenario

Course content and student records were lost after accidental deletion in the cloud. The platform needs protection against human error as well as failure of a zone or region, with defined recovery targets and a tested continuity plan.

2. Risks and issues identified

- **Accidental or malicious deletion** by users with excessive permissions; deletions that replicate instantly to copies.
- **No versioning or point-in-time recovery**, and backups stored beside the data they protect.
- **Zone/region outage, ransomware, corruption**, and **untested restores** (backups that cannot be restored).

3. Application of concepts: proposed solution

- **Versioning:** object-storage versioning plus MFA-delete and soft-delete/recycle bin so deleted or overwritten objects can be recovered.
- **Cloud-native backup:** centrally managed backup plans (for example AWS Backup/Azure Backup) with automated hourly snapshots, daily and weekly retention, and database point-in-time recovery (PITR).
- **Replication:** cross-region replication of objects, snapshots and the database to a DR region, with copies in a separate account/vault using **object lock (WORM)** so ransomware or an admin error cannot remove them.
- **3-2-1 rule:** 3 copies, 2 storage types, 1 off-site/immutable copy; least-privilege IAM and separate delete permissions.

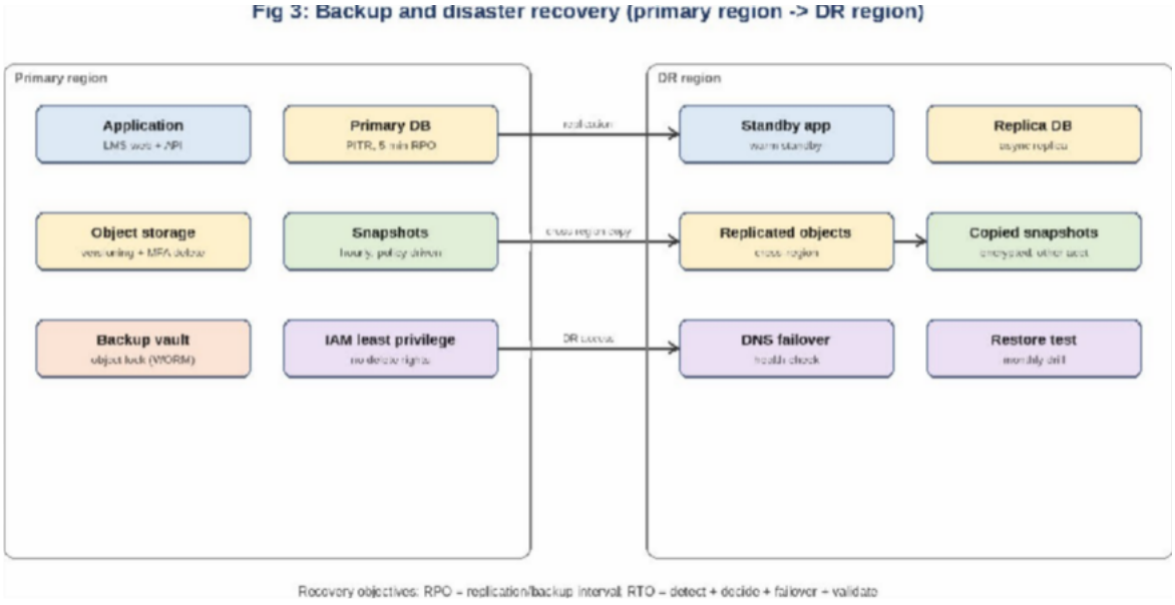


Figure 3: Backup, versioning and cross-region replication to a DR region.

4. How RTO and RPO are achieved

DR strategy	Typical RPO	Typical RTO	Comment
Backup and restore	Hours	Hours	Cheapest; restore time grows with data size
Pilot light	Minutes	10s of minutes	Core data replicated; compute started on demand
Warm standby	Seconds–minutes	Minutes	Scaled-down copy always running (chosen for LMS)
Active-active	Near zero	Near zero	Most expensive

RPO is met by how often data is captured: hourly snapshots give RPO ≤ 60 min, database PITR about 5 min, and continuous cross-region replication under 1 min. **RTO** is met by pre-provisioned standby compute, infrastructure as code, automated DNS failover and runbooks.

5. Calculations (assumed values)

Item	Calculation and result
Restore time from backup	5 TB = 5 × 1,024 × 1,024 = 5,242,880 MB; at 500 MB/s → 10,486 s ≈ **2.9 hours** (too long for the target, hence a warm standby)
Target RTO with warm standby	Detect 2 min + decide 3 min + DNS/failover 5 min + validate 5 min = **15 min**
Storage with versioning (30-day retention)	Base 5,120 GB × $0.023 = $117.76. Versions: 5,120 × 10% daily change × 30 = 15,360 GB × $0.023 = $353.28. Total **$471.04/month**
With lifecycle to infrequent access ($0.0125/GB)	Old versions: 15,360 × 0.0125 = $192.00. Total 117.76 + 192.00 = **$309.76/month**, saving **34%**

6. Business continuity steps and justification

1. **Detect:** monitoring and alarms identify the failure or data loss.
2. **Declare and assess:** the incident lead classifies severity and decides recovery versus failover.
3. **Recover:** restore the deleted object from a version, or fail over DNS to the DR region and promote the replica database.
4. **Validate:** run smoke tests, check data integrity and sessions before opening to all users.
5. **Communicate:** notify students, staff and management through status page and email.
6. **Fail back and review:** return to the primary region after repair; hold a post-incident review and update the runbook; test restores monthly.

Justification: versioning plus immutable backups directly solve the accidental-deletion cause, while a warm standby brings the RTO down from about 2.9 h to about 15 min at moderate cost, which suits an education platform better than costly active-active.

Q4. Hybrid Cloud for a Government Agency

1. Understanding of the scenario

The agency has sensitive data and applications that compliance rules require to stay on-premises, and non-sensitive workloads that can use public cloud for elasticity. The two environments must be connected securely under one identity, encryption and governance model.

2. Risks and issues identified

- **Data leakage** across the boundary, or sensitive data accidentally placed in public cloud.
- **Insecure links** (public internet), weak or duplicated identities, key mismanagement.
- **Compliance and audit gaps**, inconsistent policies, limited visibility, latency and integration problems.

3. Application of concepts: hybrid architecture

- **Data placement:** classify data; sensitive systems stay in the on-premises data centre, non-sensitive and public-facing workloads run in the public cloud; only approved, anonymised data flows outward.
- **Secure connectivity:** private dedicated link (Direct Connect/ExpressRoute) with an IPsec site-to-site VPN as backup, hub-and-spoke transit network, firewalls/IDS at the boundary, network segmentation, private endpoints and no public IPs for internal services.

- **Zero-trust:** every request is authenticated, authorised and encrypted regardless of origin; micro-segmentation limits lateral movement.

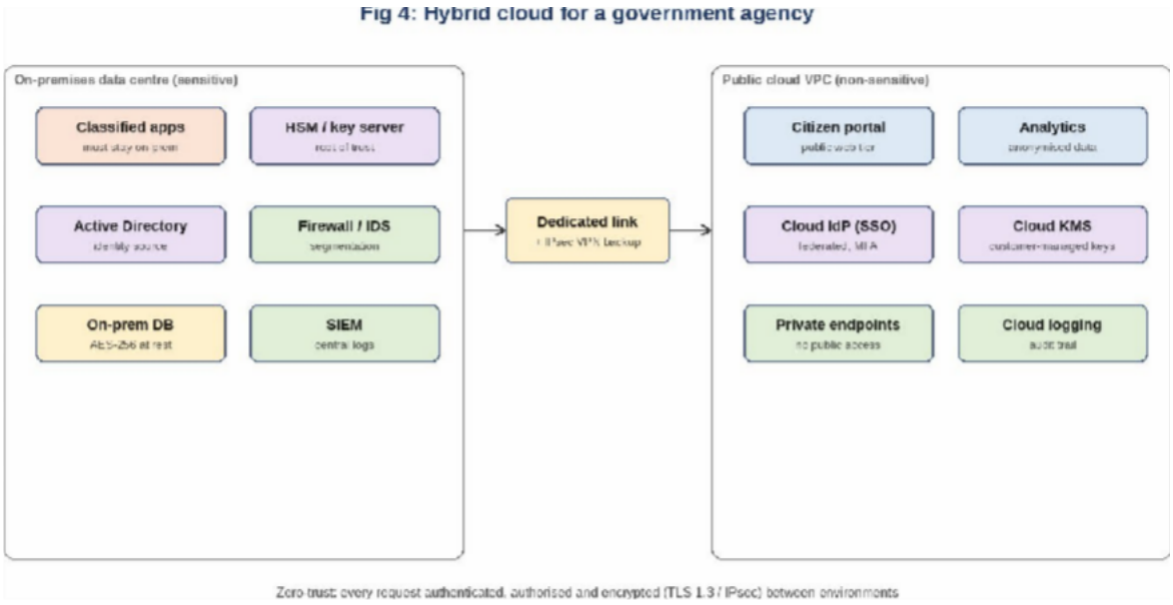


Figure 4: Hybrid architecture linking on-premises and public cloud over private, encrypted connectivity.

4. Identity management and encryption

Area	Implementation
Identity	Single source of truth in on-premises Active Directory, federated to the cloud IdP using SAML 2.0/OIDC for **SSO**; MFA for all users; RBAC/ABAC with least privilege; privileged access management with just-in-time access; automated joiner/mover/leaver provisioning (SCIM); conditional access.
Encryption at rest	AES-256 for databases, disks and object storage; keys held in an on-premises **HSM** or cloud KMS with customer-managed keys (BYOK/HYOK for the most sensitive data); rotation and separation of duties.
Encryption in transit	TLS 1.3 for applications, IPsec/MACsec on links, mutual TLS between services; certificate management.
Monitoring	Central SIEM collecting on-prem and cloud logs; key-use and admin-action auditing.

5. Calculations (assumed values)

Item	Calculation and result
Average sync bandwidth	2 TB/day = $2 \times 10^{12} \times 8 = 16 \times 10^{12}$ bits / 86,400 s ≈ **185 Mbps**
Peak bandwidth (3× average)	185 × 3 ≈ **556 Mbps** → choose a **1 Gbps** dedicated link (peak utilisation 55.6%), with VPN backup

Item	Calculation and result
Initial 10 TB migration time	10 TB = 80 × 10^{12} bits; usable 1 Gbps × 80% = 0.8 Gbps → 100,000 s ≈ **27.8 hours**
Connectivity availability	Dedicated 99.9% and VPN 99.5%: combined = 1 − (0.001 × 0.005) = **99.9995%**

6. Challenges and decision justification

Challenge	How it is managed
Consistent security policy	Policy-as-code, common baselines, central identity
Visibility and monitoring	Unified SIEM, logging and observability across both sides
Latency and bandwidth	Dedicated link, caching, keep chatty apps on the same side
Skills and tool sprawl	Standard tooling (Terraform, Kubernetes), training, runbooks
Compliance and data sovereignty	Data classification, residency controls, regular audits
Cost and integration	FinOps tagging, API gateways, phased migration

Justification: a private dedicated link is chosen over internet VPN alone because of predictable latency and security for government data; the calculation shows 1 Gbps comfortably covers the peak load; customer-controlled keys and federated identity keep control with the agency while still using cloud elasticity.

Q5. High Availability and Fault Tolerance for a SaaS Provider

1. Understanding of the scenario

The SaaS application must keep running even if an entire cloud region fails. That requires redundancy at every layer (compute, data and network), automated failover, and monitoring that detects problems before customers are affected.

2. Risks and issues identified

- **Single points of failure:** a single region, a single database, one DNS or load balancer.
- **Data loss or inconsistency** during failover because of replication lag.
- **Slow, manual failover**, capacity shortfall in the surviving region, and failover that was never tested.
- **Late detection** of degradation (no alerting on user-facing symptoms).

3. Application of concepts: multi-region deployment strategy

- **Active-active in two or more regions**, each with a full stack spread across at least 3 availability zones; stateless application tier in auto-scaling groups.
- **Global traffic management:** DNS/Anycast with latency-based routing and health checks directs users to the nearest healthy region and removes a failed region automatically.
- **Infrastructure as code** keeps regions identical; deployments are rolled out region by region (canary/blue-green).

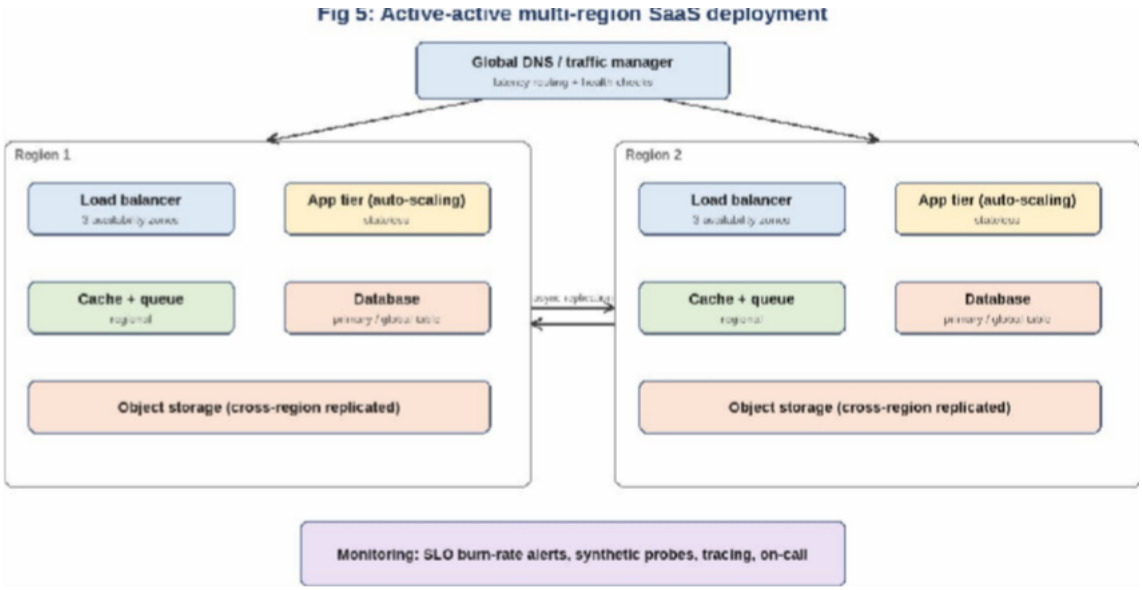


Figure 5: Active-active multi-region deployment with global traffic management and monitoring.

4. Data replication and failover configuration

Component	Configuration
Database	Managed global database (for example Aurora Global/DynamoDB global tables/Cosmos DB) with asynchronous cross-region replication (lag about 1 s); a replica can be promoted automatically; conflict resolution rules for multi-writer designs.
Object storage	Cross-region replication with versioning; replicated encryption keys per region.
Cache and queues	Regional caches rebuilt from the database; replicated or regional queues with idempotent consumers.
Failover	Health checks every 10 s, failure after 3 misses, DNS TTL 60 s; automatic removal of the failed region; scripted promotion of the database replica; runbook-driven failback.
Testing	Regular game days and chaos tests (region isolation drills) to prove RTO and RPO.

5. Calculations (assumed values)

Item	Calculation and result
Single region (99.95%)	0.0005 × 8,760 h = **4.38 h/year** downtime
Two regions active-active	$1 - 0.0005^2 =$ **99.999975%** → about **7.9 s/year** (theoretical; shared dependencies reduce it in practice)
Worst-case failover time (RTO)	Detection 10 s × 3 = 30 s + DNS TTL 60 s = **90 s**; with manual database promotion (+60 s) ≈ 150 s
RPO	Equal to replication lag ≈ **1 s** of data

Item	Calculation and result
Capacity headroom, 2 regions	Each must carry 100% after a failure, so steady load ≤ **50%** per region
Capacity headroom, 3 regions	After losing one, each remaining region carries 1/2 of the load; steady load per region = 1/3, so utilisation = 33.3 / 50 = **66.7%** (more efficient than 2 regions)
Error budget, SLO 99.99%	0.0001 × 525,600 min = **52.6 min/year** (SLO 99.9% = 525.6 min = 8.76 h)

6. Monitoring and alerting; decision justification

- **Metrics:** the four golden signals (latency, traffic, errors, saturation) per region, replication lag, queue depth, and health-check status.
- **Logs and tracing:** centralised structured logs and distributed tracing to find the failing component quickly.
- **Synthetic monitoring:** probes from several regions simulate user journeys and detect failures before customers report them.
- **Alerting:** SLO burn-rate alerts (for example a 14.4× burn over 1 h uses 2% of a 30-day budget and pages on-call), tiered severities, runbook links and automated remediation; dashboards for each region.
- **Proactive detection:** anomaly detection on trends, capacity forecasts, and regular failover drills.

Justification: active-active across regions is chosen because it turns a region loss into a brief traffic shift (about 90 s) rather than an outage; the calculation shows availability rising from 99.95% to about 99.9999%. Three regions are more cost-efficient than two for the same protection, and SLO-based alerting ensures problems are caught early.